

AI Model Used: **Gemini, Claude (in Antigravity IDE)**

What AI Was Used For:

AI was used for:

- Project planning, breaking down requirements, finding and selecting node.js libraries
- Getting started on the code for the RSA key generation in keyManager.js and the Express server in server.js
- Refactoring and updating my code based on the gradebot test client results
- Generating most of the test suite to meet the coverage requirements
- Structuring, updating, and revising the README
- Revising git commit messages

Prompt 1:

"Implementing a basic JWKS Server

Objective

Develop a RESTful JWKS server that provides public keys with unique identifiers (kid) for verifying JSON Web Tokens (JWTs), implements key expiry for enhanced security, includes an authentication endpoint, and handles the issuance of JWTs with expired keys based on a query parameter.

Chooses an appropriate language and web server for the task.

Due to the simplicity of this assignment, I would prefer you complete it with an unfamiliar language... but as I have no way to verify it, it's not considered part of the rubric.

This project is for educational purposes. In a real-world scenario, you'd want to integrate with a proper authentication system and ensure security best practices.

Background

HTTP/web services

Familiarize yourself with client/server HTTP services.

REST

Familiarize yourself with correct HTTP methods/headers/status codes for RESTful APIs.

JOSE: JWT, JWK (and JWKS):

Familiarize yourself with the concepts of JWT, JWK.

Understand the importance of key expiry, and kid.

Requirements

Key Generation

Implement RSA key pair generation.

Associate a Key ID (kid) and expiry timestamp with each key.

Web server with two handlers

Serve HTTP on port 8080

A RESTful JWKS endpoint that serves the public keys in JWKS format. Only serve keys that have not expired.

The endpoint should be the idiomatic well-known JWKS URI.

A /auth endpoint that returns an unexpired, signed JWT on a POST request. If the “expired” query parameter is present, issue a JWT signed with the expired key pair and the expired expiry.

Documentation

Code should be organized.

Code should be commented where needed.

Code should be linted per your language/framework.

Tests

Test suite for your given language/framework with tests for you.

Test coverage should be over 80%.

Blackbox testing

Ensure the included test client functions against your server.

The testing client will attempt a POST to /auth with no body. There is no need to check authentication for this project. NOTE: We are not actually testing user authentication, just mocking authentication and returning a valid JWT for this user

Note:

Using kid in JWKS is crucial for systems to identify which key to use for JWT verification. Ensure that the JWTs include the kid in their headers and that the JWKS server can serve the correct key when requested with a specific kid.

Expected Outcome

At the end of the project, you should have a functional JWKS server with a RESTful API that can serve public keys with expiry and unique kid to verify JWTs.

The server should authenticate fake users requests, issue JWTs upon successful authentication, and handle the “expired” query parameter to issue JWTs signed with an expired key.

This project should take 1-12 hours, depending on your familiarity with your chosen language/framework, and web servers in general.

Deliverables

Provide a link to your GitHub repo containing your code. Include in the repo a screenshot of the test client running against your server.

Include in the repo a screenshot of your test suite (if present) showing the coverage percent."

Can you help me understand what this project does? I'm not familiar with JOSE, and i've never coded anything involve REST before. What is the purpose of the public and private keys here? What does it mean to "serve" HTTP on port 8080, or "serve" public keys in JWKS format? What should happen when i run the completed program and what's the point?

Prompt 2:

I usually use python for assignments because it's easy but i want to try using JavaScript and Node.js because I'm interested in learning more about those. Do i have to use express.js as well? Can you help me get started on this project? What files do I create? My first task is just to implement the rsa key pair generation and associate a key id and expiry timestamp with each key.

Prompt 2:

```
PS C:\Users\mxz\Documents\College\YEAR 4\CSCE 3550\jwks-server> node -v
v26.8.2
PS C:\Users\mxz\Documents\College\YEAR 4\CSCE 3550\jwks-server> npm init -y
npm : File C:\Program Files\nodejs\npm.ps1 cannot be loaded because running scripts is
disabled on this system. For more information, see about_Execution_Policies at
https://go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:1
+ npm init -y
+ ~~~
+ CategoryInfo          : SecurityError: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
PS C:\Users\mxz\Documents\College\YEAR 4\CSCE 3550\jwks-server>
```

Prompt 3:

```
"const crypto = require("crypto");

// Array to store generated keys
const keys = [];

// Function to generate key data
function generateKey(expired = false) {
  // Generate an RSA key pair
  const { publicKey, privateKey } = crypto.generateKeyPairSync("rsa", {
    modulusLength: 2048,
    publicKeyEncoding: {
      type: 'spki',
      format: 'jwk',
    },
    privateKeyEncoding: {
      type: 'pkcs1',
      format: 'pem',
    }
  });

  // Create a unique Key ID (kid)
  const kid = crypto.randomUUID();

  // Set expiry timestamp (1 hour from now or 1 hour ago if expired)
  const expiryOffset = expired ? -3600 : 3600;
  const expiry = Math.floor(Date.now() / 1000) + expiryOffset;

  // Combine the data into a single object
  const keyData = {
```

```

    kid: kid,
    expiry: expiry,
    publicKey: {
      ...publicKey,
      kid: kid,
      alg: 'RS256',
      use: 'sig',
    },
    privateKey: privateKey
  };

  // Store the key in the array
  keys.push(keyData);
  return keyData;
}

// Generate initial keys, one valid and one expired
generateKey();
generateKey(true);

module.exports = {
  keys,
  generateKey,
};

console.log(JSON.stringify(keys, null, 2));"

```

Can you create a simple brief commit summary and description for the changes to keymanager.js

Prompt 4:

Now i need your help with server.js. Can you still read the original project instructions i attached at the beginning? Here's what i want to do:

Implement an Express.js/Node.js server that listens on port 8080

First endpoint: When someone visits a specific URL on the server, (e.g., /.well-known/jwks.json), the server should respond by sending them a JSON file containing the public keys (JWKs)

Second endpoint: When a POST request is sent to http://localhost:8080/auth, the server should respond by generating a JWT, signing it with the private key, and sending it back

What libraries do I use and how do i get started on this

Prompt 5:

```
PS C:\Users\mxz\Documents\College\YEAR 4\CSCE 3550\jwks-server> curl -X POST
http://localhost:8080/auth
Invoke-WebRequest : A parameter cannot be found that matches parameter name 'X'.
At line:1 char:6
+ curl -X POST http://localhost:8080/auth
+ ~~~
+ CategoryInfo          : InvalidArgument: (:) [Invoke-WebRequest],
ParameterBindingException
+ FullyQualifiedErrorId :
NamedParameterNotFound,Microsoft.PowerShell.Commands.InvokeWebRequestCommand

PS C:\Users\mxz\Documents\College\YEAR 4\CSCE 3550\jwks-server>
```

Prompt 6:

review/reivse this commit summary and maybe add a description if necessary:
Implement web server with JWKS GET endpoint and /auth POST endpoint

Prompt 7:

Conversation with Gemini
"Implementing a basic JWKS Server

Objective

Develop a RESTful JWKS server that provides public keys with unique identifiers (kid) for verifying JSON Web Tokens (JWTs), implements key expiry for enhanced security, includes an authentication endpoint, and handles the issuance of JWTs with expired keys based on a query parameter.

Chooses an appropriate language and web server for the task.

Due to the simplicity of this assignment, I would prefer you complete it with an unfamiliar language... but as I have no way to verify it, it's not considered part of the rubric.

This project is for educational purposes. In a real-world scenario, you'd want to integrate with a proper authentication system and ensure security best practices.

Background

HTTP/web services

Familiarize yourself with client/server HTTP services.

REST

Familiarize yourself with correct HTTP methods/headers/status codes for RESTful APIs.

JOSE: JWT, JWK (and JWKS):

Familiarize yourself with the concepts of JWT, JWK.

Understand the importance of key expiry, and kid.

Requirements

Key Generation

Implement RSA key pair generation.

Associate a Key ID (kid) and expiry timestamp with each key.

Web server with two handlers

Serve HTTP on port 8080

A RESTful JWKS endpoint that serves the public keys in JWKS format. Only serve keys that have not expired.

A /auth endpoint that returns an unexpired, signed JWT on a POST request. If the “expired” query parameter is present, issue a JWT signed with the expired key pair and the expired expiry.

Documentation

Code should be organized.

Code should be commented where needed.

Code should be linted per your language/framework.

Tests

Test suite for your given language/framework with tests for you.

Test coverage should be over 80%.

Blackbox testing

Ensure the included test client functions against your server.

The testing client will attempt a POST to /auth with no body. There is no need to check authentication for this project. NOTE: We are not actually testing user authentication, just mocking authentication and returning a valid JWT for this user

Note:

Using kid in JWKS is crucial for systems to identify which key to use for JWT verification. Ensure that the JWTs include the kid in their headers and that the JWKS server can serve the correct key when requested with a specific kid.

Expected Outcome

At the end of the project, you should have a functional JWKS server with a RESTful API that can serve public keys with expiry and unique kid to verify JWTs.

The server should authenticate fake users requests, issue JWTs upon successful authentication, and handle the “expired” query parameter to issue JWTs signed with an expired key.

This project should take 1-12 hours, depending on your familiarity with your chosen language/framework, and web servers in general.

Deliverables

Provide a link to your GitHub repo containing your code. Include in the repo a screenshot of the test client running against your server.

Include in the repo a screenshot of your test suite (if present) showing the coverage percent.

As always with every screenshot, please include identifying information.

Project 1

CriteriaRatingsPointsValid JWT authentication

[view longer description](#)

Full Marks

15 pts

No Marks

0 pts

/15 pts

Expired JWT authentication

[view longer description](#)

Full Marks

5 pts

No Marks

0 pts

/5 pts

Proper HTTP Methods And Status Codes

[view longer description](#)

Full Marks

10 pts

No Marks

0 pts

/10 pts

Valid JWK Found In JWKS

[view longer description](#)

Full Marks

20 pts

No Marks

0 pts

/20 pts

Expired JWK is expired

[view longer description](#)

Full Marks

5 pts

No Marks

0 pts

/5 pts

Expired JWK Not Found In JWKS

[view longer description](#)

Full Marks

10 pts

No Marks

0 pts

/10 pts

Test suite is present

[view longer description](#)

Full Marks

15 pts

No Marks

0 pts

/15 pts

Test coverage

[view longer description](#)

Full Marks

5 pts

No Marks

0 pts

/5 pts

Documentation/Linting/Organization

[view longer description](#)

Full Marks

5 pts

No Marks

0 pts

/5 pts"

These are the assignment instructions for my jwks server project for my foundations of cybersecurity class. I've already written code for server.js and keyManager.js. I ran the gradebot client using `./gradebot project-1 --port 8080 --run "node server.js"` and got these results:"

NAME	POINTS	AWARDED	NOTES
/auth valid JWT authN	15.00	15.00	
/auth?expired=true JWT authN (expired)	5.00	5.00	
Proper HTTP methods/Status codes	10.00	1.00	
Valid JWK found in JWKS	20.00	20.00	
Expired JWT is expired	5.00	5.00	
Expired JWK does not exist in JWKS	10.00	10.00	
Quality benefit	5.00	3.75	The code is generally well-organized, but could
			from clearer separation of concerns, such as moving
key			generation logic to a dedicated module.

		Best practices are mostly followed, but consider using
		environment variables for sensitive configurations and
		validating input in the endpoints.
		Readability is good, but adding more comments and
understanding.		JSDoc-style documentation would enhance
		Overall, the code quality is solid, but ensure that error
		handling is consistent and consider implementing rate
		limiting for the endpoints.
		The JWT implementation is functional, but ensure that
in logs		private keys are securely managed and not exposed
		or responses.
746d28fc-bca9-453c-892c-e911170b5931 Grade: 85.36%		

Can you help me update my code based on the notes from the gradebot?

Prompt 8:

"PS C:\Users\mxz\Documents\College\YEAR 4\CSCE 3550\jwks-server> ./gradebot project-1 --port 8080 --run "node server.js"			
NAME	POINTS	AWARDED	NOTES
/auth valid JWT authN	15.00	15.00	
/auth?expired=true JWT authN (expired)	5.00	5.00	

[illegible]

Submit results to server? (y/n): n

PS C:\Users\mxz\Documents\College\YEAR 4\CSCE 3550\jwks-server> "

Why am i still only getting 1/10 points for proper http methods/status codes?

Prompt 9:

The gradebot now gave me 10/10 points for the http method/status codes. So is it not possible to receive those points using Express?

Prompt 10:

Ok first, can you give me a brief commit summary and description. I'm going to make a commit for all the changes you made to keyManager.js, package.json, and server.js since i first told you to update the code based on the gradebot results. Then, after I commit the current code, I want you to try to use Express 4 to make the code more similar to how it was before (since it was simpler and easier to understand before) but still have it work with the gradebot.

Prompt 11:

Now give me a brief commit summary and description for the changes you just made (switching back to express code with express 4)

Prompt 12:

I'm pretty much done with everything except for the test suite. I made two files for the test suite. Can you help me make the test suite for the program?

Prompt 13:

Ok now finally, can you help me update the README? Make sure to include sections like the tech stack/technologies used, instructions on how to run, the prerequisites/dependencies needed, how to run the test suite, how to get the program onto your computer, etc. I needed to download node.js, i used Express for the server and jsonwebtoken to make and sign the JWTs, I run the server using "node server.js," and I run the test suite using "npm test". Also, include these commands for manual testing:

```
curl.exe http://localhost:8080/.well-known/jwks.json
```

```
curl.exe -X POST http://localhost:8080/auth
```

```
curl.exe -X POST "http://localhost:8080/auth?expired=true"
```

And mention that you can verify the JWTs using jwt.io

Prompt 14:

Can you make a brief commit message for all the changes made AFTER making the test suite files. DO NOT mention the readme updates though, i'll make a separate commit for those.

Basically, I want a commit message mentioning the addition of all the code related to the test suites

Prompt 15:

review/revise this commit summary:

Update README with instructions on how to run the test client against the server and the screenshots of the test client and teste suite results inside /screenshots